

MATLAB

1. Úvod do MATLABu a dátové typy



Stredná priemyselná škola elektrotechnická, Karola Adlera 5

Čo nás dnes čaká

1. Spustenie MATLABu a jeho prostredie
2. Premenné a priradovací príkaz
3. Dátové typy — ako fungujú a prečo na nich záleží
4. Rozsahy typov, pretypovanie a saturácia
5. Znaky a reťazce
6. Relačné a logické výrazy
7. Vstavané funkcie a ukladanie premenných

Spustenie MATLABu

- Po spustení sa otvorí **Command Window** so znakom výzvy (prompt): >>
- Za prompt píšeme príkaz a MATLAB **okamžite** odpovie výsledkom – interaktívny režim
- Neskôr budeme písať aj **skripty** – súbory .m s postupnosťou príkazov
- Pomoc: `help` funkcia (stručne), `doc` funkcia (podrobne s príkladmi)
- `lookfor` slovo hľadá v nápovede podľa slova, `demo` otvorí ukážky
- Ukončenie: `quit` alebo `exit`

Prostredie MATLABu (Desktop)

- **Command Window** — príkazový riadok, srdce práce s MATLABom
- **Workspace** — zoznam premenných, ktoré práve existujú v pamäti, aj ich hodnoty
- **Current Folder** — aktuálny priečinok; sem sa ukladajú súbory (dôležité nastaviť správne!)
- **Command History** — história zadanych príkazov, aj z predchádzajúcich sedení
- Hore je pás nástrojov so záložkami **HOME**, **PLOTS**, **APPS**
- Užitočné: `clc` vyčistí okno, `clear` zmaže premenné

Premenné a priradovanie — teória

- Tvar príkazu: `nazov_premennej = vyraz`
- Znak `=` je **priradovací operátor**, nie matematická rovnosť — čítame ho ako „*premenná dostane hodnotu*“
- Premenná musí byť **vľavo**; `6 = mynum` skončí chybou
- Bodkočiarka `;` na konci **potlačí výpis** (výpočet sa aj tak vykoná)
- Ak výsledok nepriradíme, MATLAB ho uloží do premennej `ans`
- Šípka **hore** vyvolá predchádzajúci príkaz — netreba prepisovať dlhé výrazy

Premenné a priradovanie — príklad

```
mynum = 3  
mynum = 4 + 2  
mynum = mynum + 1  
6 + 3
```

```
mynum =  
3  
  
mynum =  
6  
  
mynum =  
7  
  
ans =  
9
```

Riadok `mynum = mynum + 1` nie je rovnica — vezme starú hodnotu, pripočíta 1 a uloží späť.

Názvy premenných a pracovný priestor

- Názov musí začínať **písmenom**, ďalej môže obsahovať písmená, číslice a podčiarkovník
- **Bez medzier** a bez diakritiky; MATLAB rozlišuje veľké a malé písmená (mynum $\neq$ MYNUM)
- Nepoužívame kľúčové slová (if, for, end, ...) ani názvy vstavaných funkcií
- Názov má byť **výstižný**: radius je lepšie ako x
- `who` vypíše mená premenných, `whos` aj typ a veľkosť v pamäti
- `clear` premenna zmaže jednu, `clear` všetky

Dátové typy — prečo a ako

- Typ premennej **nedeklarujeme** (na rozdiel od C) — MATLAB si ho odvodí sám pri priradení
- Napriek tomu **každá premenná typ má**; v MATLABe sa typ nazýva aj **trieda** (class)
- Zistíme ho funkciou `class(x)`
- Predvolený číselný typ je `double` — aj celé čísla sú `double`, pokiaľ nepoviemme inak
- Prečo na tom záleží: **pamäť** (double zaberá 8 bajtov, int32 iba 4), **presnosť** výpočtu, **rozsah** hodnôt a kompatibilita s funkciami

Prehľad dátových typov

Skupina	Typy	Použitie
Desatinné čísla	double, single	bežné výpočty
Celé čísla so znamienkom	int8, int16, int32, int64	úspora pamäte
Celé čísla bez znamienka	uint8, ..., uint64	len nezáporné hodnoty
Znaky a text	char, string	jednoduché vs dvojité úvodzovky
Logický typ	logical	výsledok porovnania

Číslo v názve udáva počet **bitov**: int8 používa 8 bitov, teda 1 bajt.

Dátové typy – príklad

```
num = 6 + 3;  
class(num)  
numi = int32(num);  
whos num numi
```

```
ans =  
  
    'double'  
  
Name    Size    Bytes  Class  
  
num      1x1         8  double  
numi     1x1         4  int32
```

Tá istá hodnota 9 – ale ako `double` zaberá 8 bajtov a ako `int32` len 4.

Rozsahy typov a pretypovanie – teória

- Každý celočíselný typ má **obmedzený rozsah**: `int8` od -128 do 127 , `uint8` od 0 do 255
- Hranice zistíme funkciami `intmin('int8')` a `intmax('int8')`
- **Pretypovanie** (type casting): meno typu použijeme ako funkciu, napr. `int32(x)`
- Ak sa hodnota do typu **nezmestí**, MATLAB ju **oreže na najbližšiu hranicu** – tzv. **saturačná aritmetika**
- V jazyku C by sa hodnota „pretočila“ na druhú stranu rozsahu; MATLAB to **nerobí**, čím chráni pred tichými chybami

Rozsahy typov a saturácia — príklad

```
intmin('int8')  
intmax('int8')  
int8(200)  
uint8(-5)
```

```
ans =  
    int8  
   -128
```

```
ans =  
    int8  
    127
```

```
ans =  
    int8  
    127
```

```
ans =  
    uint8  
         0
```

Znaky a reťazce — teória

- `'ahoj'` v jednoduchých úvodzovkách → typ `char` (pole znakov)
- `"ahoj"` v dvojitých úvodzovkách → typ `string` (novší textový typ)
- Znaky sú v počítači uložené ako **čísla** podľa kódovania znakov (prvých 128 = ASCII)
- `double('a')` dá číselnú hodnotu znaku, `char(97)` opačne
- Písmená idú **za sebou**: `'a'`=97, `'b'`=98, `'c'`=99 — preto sa dajú porovnávať aj počítat

Znaky a reťazce — príklad

```
myword = 'hello'  
class(myword)  
yourword = "ciao"  
double('a')  
char('abcd' + 1)
```

```
myword =  
    'hello'  
  
ans =  
    'char'  
  
yourword =  
    "ciao"  
  
ans =  
    97  
  
ans =  
    'bcde'
```

Formát výpisu čísel

```
format long
```

```
77/888
```

```
format short
```

```
77/888
```

```
ans =
```

```
0.086711711711712
```

```
ans =
```

```
0.0867
```

`format` mení **len zobrazenie**, nie uložení hodnotu — v pamäti je číslo stále rovnako presné. Dlhý výraz možno rozdeliť na viac riadkov pomocou troch bodiek ...

Relačné a logické výrazy — teória

- Relačné operátory: `>` `<` `>=` `<=` `==` `~=`
- Pozor: rovnosť je `==` (dve rovná sa), lebo jedno `=` už znamená priradenie
- Výsledkom je typ `logical`: `1` = pravda, `0` = nepravda
- Logické operátory: `&&` (a zároveň), `||` (alebo), `~` (negácia), `xor` (práve jedno z dvoch)
- Porovnávať možno aj znaky — podľa ich číselnej hodnoty v kódovaní

Relačné a logické výrazy – príklad

```
3 < 5  
class(3 < 5)  
'a' < 'c'  
xor(3 < 5, 'a' > 'c')
```

```
ans =  
    logical  
     1  
  
ans =  
    'logical'  
  
ans =  
    logical  
     1  
  
ans =  
    logical  
     1
```

Časté chyby — na čo si dať pozor

```
x = 7;  
3 < x < 5  
3 < x && x < 5  
cos(pi/2)  
cos(pi/2) == 0
```

```
ans =  
    logical  
     1  
  
ans =  
    logical  
     0  
  
ans =  
    6.1232e-17  
  
ans =  
    logical  
     0
```

Zápis $3 < x < 5$ je vždy pravdivý! Vyhodnotí sa zľava: $3 < 7$ dá 1, potom $1 < 5$ je tiež pravda. Správne je $3 < x \text{ \&\& } x < 5$. A pozor na **zaokrúhľovacie chyby** — $\cos(\pi/2)$ nie je presná nula.

Vstavane funkcie

- **Goniometrické:** `sin`, `cos`, `tan` (radiány), `sind`, `cosd` (stupne), inverzné `asin`, `acos`
- **Zaokrúhľovanie:** `round` (na najbližšie), `floor` (nadol), `ceil` (nahor), `fix` (k nule)
- **Zvyšok po delení:** `rem` a `mod`; `sign` vráti znamienko
- **Odmocniny a logaritmy:** `sqrt`, `nthroot`, `log` (prirodzený), `log10`, `log2`, `exp`
- **Prevody:** `deg2rad`, `rad2deg`
- **Konštanta e nemá vlastný názov** — získame ju ako `exp(1)`

Vstavané funkcie — príklad

```
nthroot(64,3)
rem(13,5)
sign(-5)
exp(1)
floor(-2.7)
fix(-2.7)
```

```
ans =
    4

ans =
    3

ans =
   -1

ans =
  2.7183

ans =
   -3

ans =
   -2
```

Rozdiel: `floor` ide vždy nadol (-3), `fix` smerom k nule (-2).

Ukladanie premenných – MAT-súbory

- MATLAB vie uložiť premenné aj s ich menami do súboru s príponou `.mat`
- `save` `mojedata` uloží **všetky** premenné z pracovného priestoru
- `save` `mojedata` `x y` uloží iba vybrané premenné
- `load` `mojedata` ich načíta späť do pracovného priestoru
- `who -file` `mojedata` ukáže, čo súbor obsahuje, bez načítania
- Súbor sa uloží do **aktuálneho priečinka** (Current Folder) – preto ho treba mať správne nastavený

Zhrnutie a nabudúce

- **Dnes:** prostredie MATLABu, premenné a priradovanie, dátové typy a prečo na nich záleží, rozsahy typov a saturácia, znaky a reťazce, relačné a logické výrazy, vstavané funkcie
- **Zapamätajte si:** = je priradenie, == je porovnanie; predvolený typ je double; pri pretypovaní hrozí saturácia
- **Nabudúce: vektory a matice** — vytváranie, rozmery a indexovanie od 1

Ďakujem za pozornosť

Otázky a diskusia
